

Feature Selection

1. Introduction

Feature selection is the process of selecting a subset of the most relevant and important features from the original dataset. In pattern recognition, datasets may contain many features, but not all of them contribute to classification. Feature selection helps in removing irrelevant and redundant features, improving the performance of the model.

2. Key Idea

- Select only **important features**
 - Remove **irrelevant and redundant features**
 - Improve model efficiency and accuracy
-

3. Need for Feature Selection

Feature selection is required because:

- High-dimensional data increases complexity
 - Some features may not be useful
 - Reduces computational cost
 - Improves classification accuracy
 - Helps avoid overfitting
-

4. Types of Feature Selection Methods

1. Filter Methods

- Select features based on statistical measures
- Independent of any machine learning model

Examples:

- Correlation
 - Chi-square
 - Information Gain
-

2. Wrapper Methods

- Use a learning algorithm to evaluate feature subsets
- More accurate but computationally expensive

Examples:

- Forward Selection
 - Backward Elimination
 - Recursive Feature Elimination
-

3. Embedded Methods

- Feature selection happens during model training
- Combines advantages of filter and wrapper methods

Examples:

- Decision Trees
 - LASSO Regression
-

5. Feature Selection Techniques

- Sequential Forward Selection (SFS)
 - Backward Selection
 - Branch and Bound Algorithm
-

6. Advantages of Feature Selection

- Reduces dimensionality
- Improves model performance
- Reduces training time
- Removes noise from data

7. Limitations

- May remove useful features if not applied properly
 - Some methods are computationally expensive
 - Requires careful selection criteria
-

8. Difference Between Feature Selection and Feature Extraction

Feature Selection	Feature Extraction
Selects existing features	Creates new features
Simple method	More complex
Example: Correlation	Example: PCA

9. Applications

Feature selection is used in:

- Image recognition
- Text classification
- Medical diagnosis
- Data mining
- Machine learning models

FEATURE EXTRACTION

SAME AS GIVEN IN UNIT 2

Problem Statement in Feature Selection and Feature Extraction

1. Introduction

In many real-world machine learning problems, datasets contain a **large number of features (dimensions)**. However:

- Not all features are useful
- Some are **irrelevant, redundant, or noisy**
- High dimensionality increases **computational cost** and reduces model performance

This leads to the need for **Feature Selection** and **Feature Extraction**.

2. Problem Statement

General Problem

Given a dataset:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

- . . . -

where each (x_i) has **many features**, the objective is to:

- Reduce the number of features
 - Retain **maximum relevant information**
 - Improve **accuracy, efficiency, and generalization**
-

3. Problem in Feature Selection

Definition

Select a **subset of original features** that are most relevant to the target.

Formal Objective

Find a subset $S \subseteq F$ such that:

- S maximizes model performance
- $|S| \ll |F|$ (fewer features)

Key Challenges

- Searching best subset is **combinatorial (NP-hard)**
 - Removing redundant and irrelevant features
 - Avoiding overfitting
-

4. Problem in Feature Extraction

Definition

Transform original features into a **new set of features** (lower dimensional space).

Formal Objective

Find a transformation:

$$Z = f(X)$$

such that:

- (Z) has lower dimension
- Important information is preserved

Key Challenges

- Designing transformation function
 - Minimizing information loss
 - Maintaining interpretability
-

5. Conceptual Difference

Aspect	Feature Selection	Feature Extraction
Approach	Select subset	Create new features
Output	Original features	Transformed features
Example	Selecting top 5 features	PCA

6. Conceptual Diagram

Feature Selection

Advanced Feature Selection Techniques

1. Supervised Techniques



1.1 Filter-based Approach

- Information gain
- Chi-square Test
- Fisher's Score
- Missing Value Ratio



1.2 Wrapper-based Approach

- Forward Selection
- Backward Selection
- Exhaustive Feature Selection
- Recursive Feature Elimination



1.3 Embedded Approach

- Regularization
- Random Forest Importance

2. Unsupervised Techniques



2.1 PCA



2.2 ICA



2.3 NMF



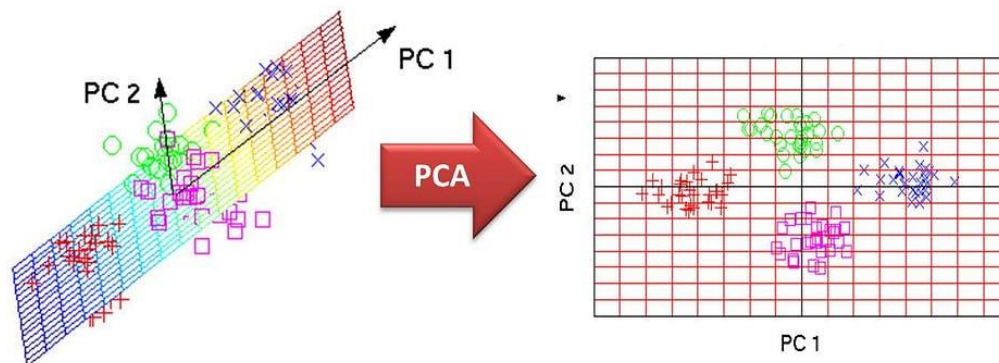
2.4 t-SNE



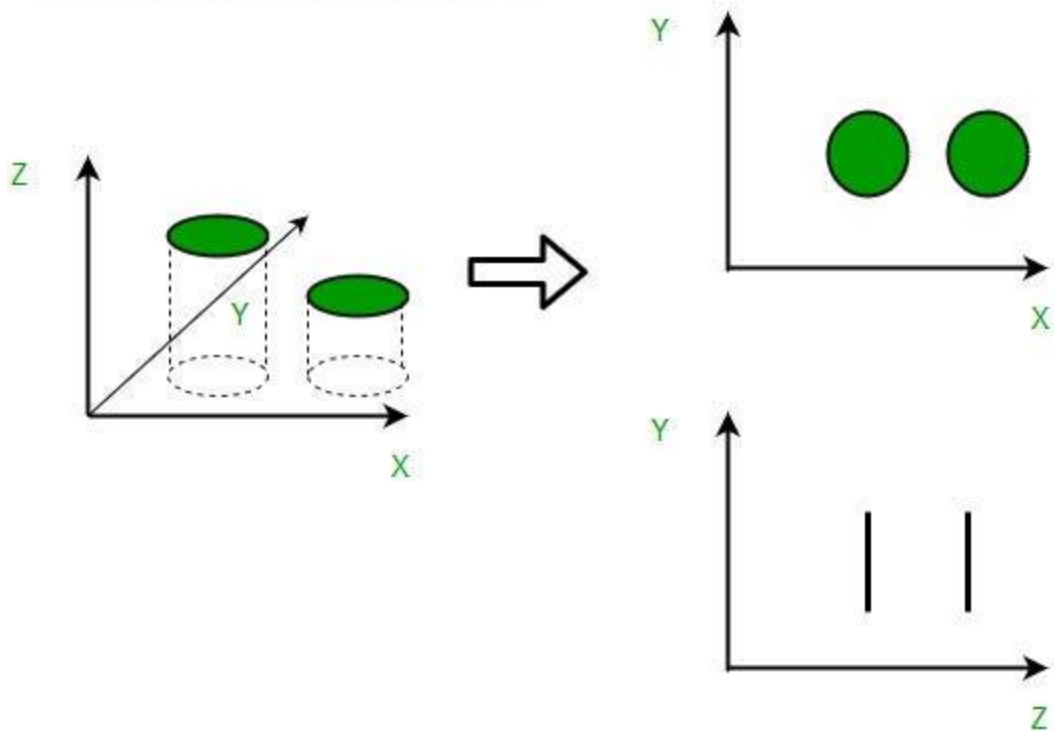
2.5 Autoencoder

Feature Extraction (Transforming Features)

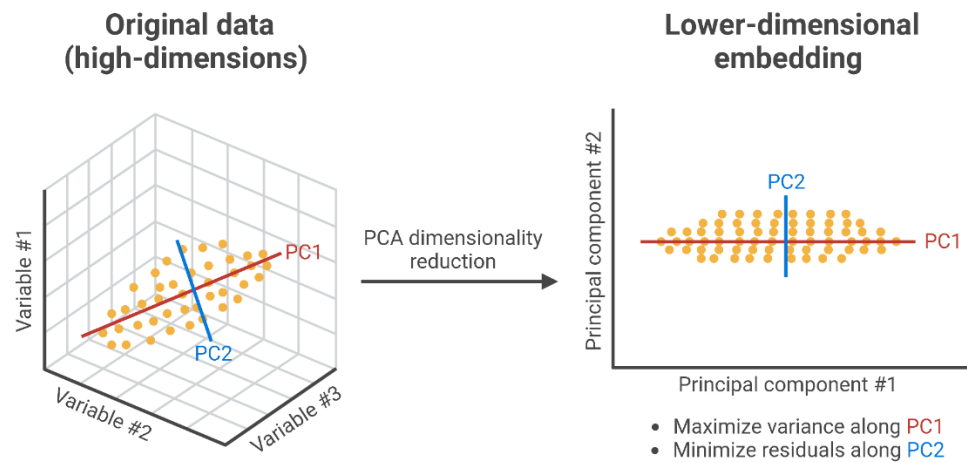
Dimensionality Reduction & Principal Component Analysis

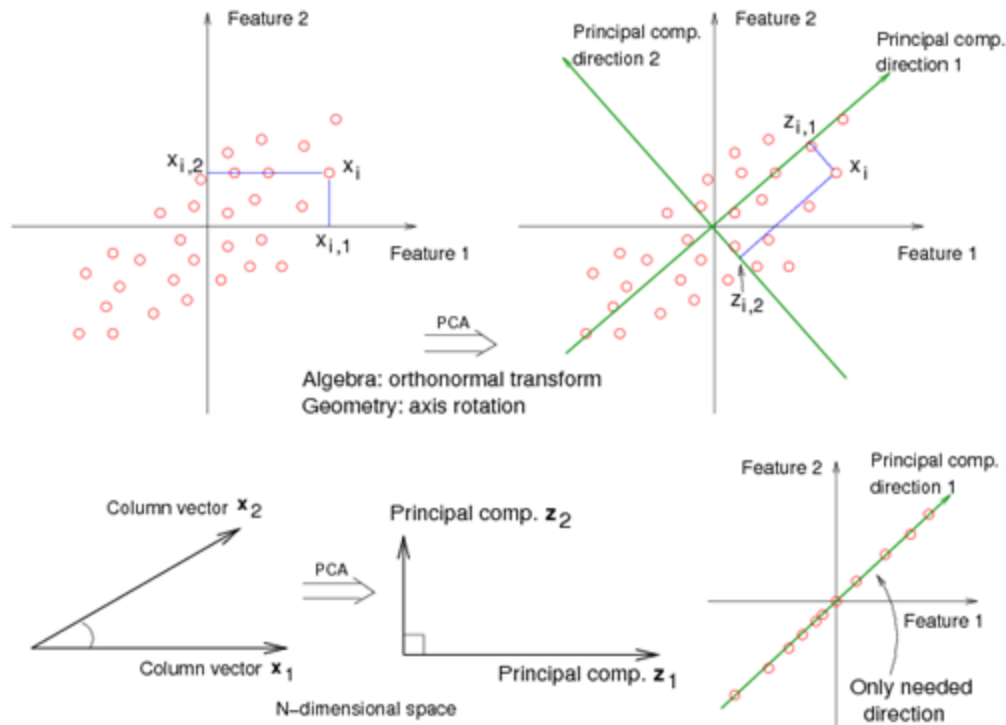


Dimensionality Reduction



Principal Component Analysis (PCA) Transformation





7. Uses / Importance

- Reduces **curse of dimensionality**
- Improves **model accuracy**
- Decreases **training time**
- Helps in **data visualization**
- Removes **noise and redundancy**

7.Summary

The **problem statement** is to reduce high-dimensional data

- Feature Selection → choose best subset
- Feature Extraction → create new compact features
- Goal → **better performance with fewer features**

Branch and Bound Algorithm

1. Introduction

Branch and Bound is an **efficient search algorithm** used to find the **optimal subset of features** without evaluating all possible combinations.

- It reduces computation by **eliminating unnecessary subsets**
 - Guarantees **optimal solution** (unlike greedy methods)
-

2. Problem Context

In feature selection:

- Given (n) features $\rightarrow$ total subsets = (2^n)
 - Exhaustive search is **computationally expensive**
 - Branch and Bound helps **prune (cut off)** unpromising subsets
-

3. Basic Idea

- **Branching** $\rightarrow$ Divide the problem into smaller subsets
- **Bounding** $\rightarrow$ Use a criterion (e.g., accuracy, error) to eliminate bad subsets

If a subset cannot give better results than the current best, it is **discarded**

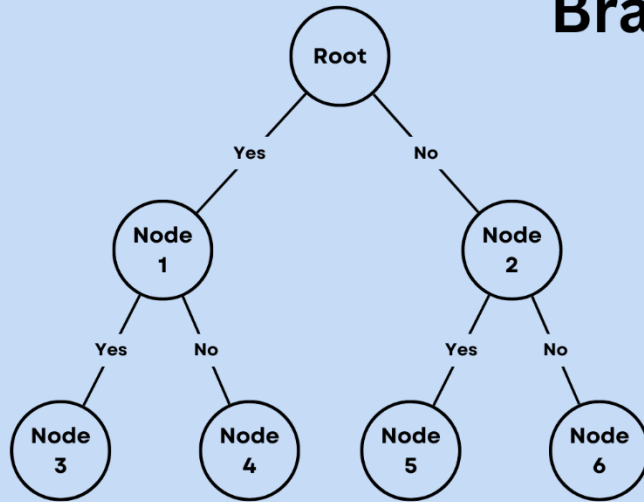
4. Working of the Algorithm

Steps

1. Start with an **empty set** or full set of features
 2. Generate subsets (branching)
 3. Evaluate subset using a **criterion function**
 4. Compare with current best solution
 5. If subset is worse $\rightarrow$ **prune it (bounding)**
 6. Continue until best subset is found
-

5. Diagram: Search Tree Representation

Branch and Bound Algorithm

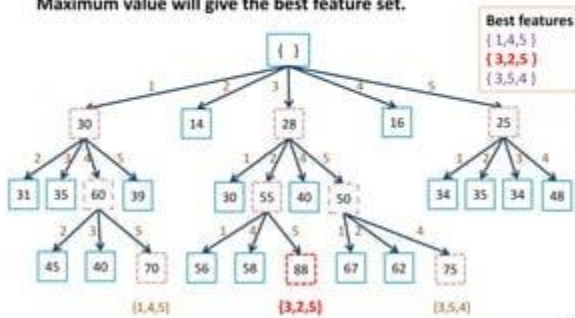


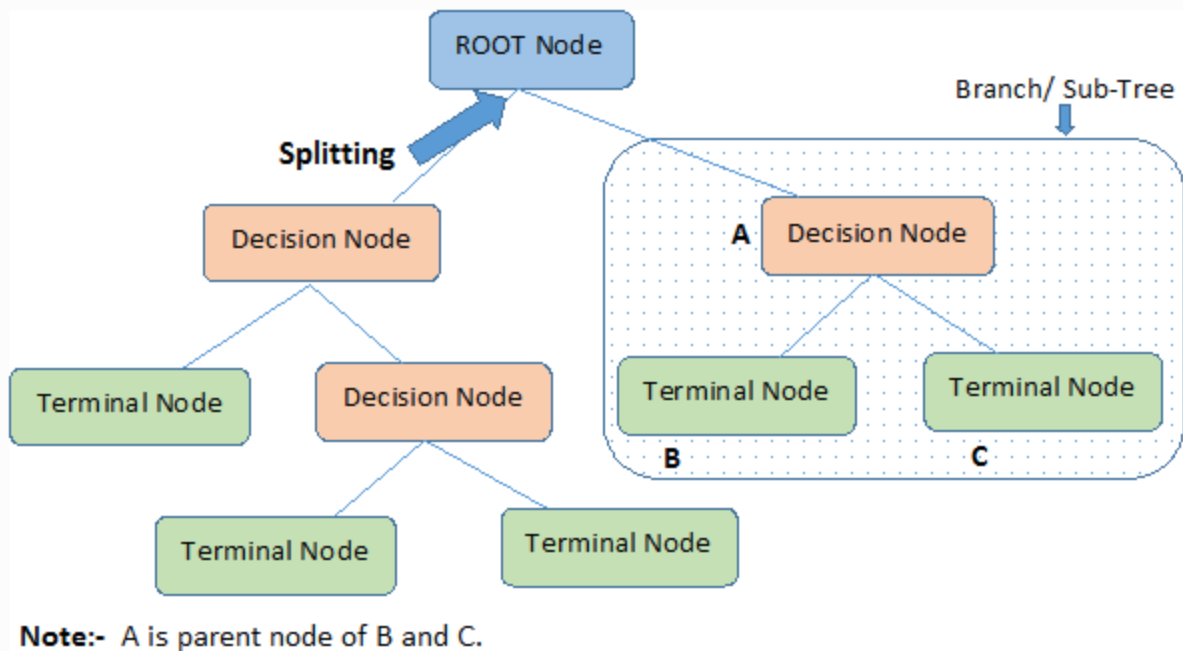
BOARD

EXAMPLE

Step 6

- Tree has reached to the 3 features which is the target subset. Maximum value will give the best feature set.





6. Key Components

1. Search Tree

- Each node represents a **subset of features**

2. Objective Function

- Measures quality (e.g., classification accuracy)

3. Bound

- A threshold to decide whether to explore further

4. Pruning

- Eliminating branches that cannot improve results
-

7. Example (Conceptual)

Suppose features = {F1, F2, F3}

- Start exploring subsets:
 - {F1}, {F2}, {F3}
 - {F1, F2}, {F1, F3}, {F2, F3}
 - If {F1, F2} performs poorly →
✗ Do not explore {F1, F2, F3}
-

8. Advantages

- Finds **optimal feature subset**
 - Reduces **search space significantly**
 - More efficient than exhaustive search
-

9. Disadvantages

- Still expensive for very large feature sets
 - Depends on quality of **bounding function**
-

10. Applications

- Feature selection in machine learning
- Pattern recognition
- Image processing
- Bioinformatics

Here are **exam-oriented notes on Sequential Forward Selection (SFS)** for B.Tech students:

Sequential Forward Selection (SFS)

1. Introduction

Sequential Forward Selection is a **greedy feature selection method** used to select the **best subset of features**.

- Starts with **no features (empty set)**
- Adds features **one by one**

- At each step, selects the feature that **improves model performance the most**
-

2. Problem Context

Given a dataset with many features:

- Goal → Select a **small subset of relevant features**
 - Avoid evaluating all (2^n) subsets
 - Use a **simple and efficient approach**
-

3. Basic Idea

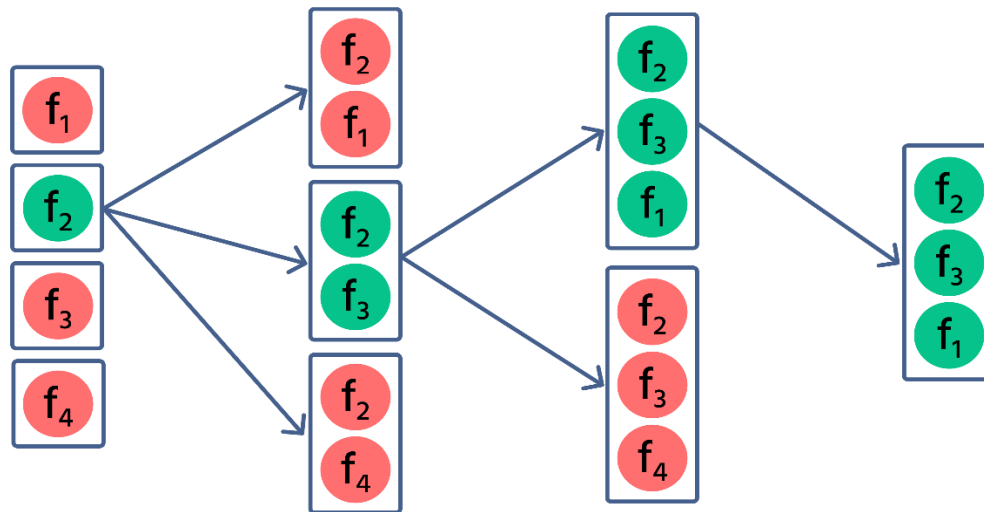
- Begin with an empty set ($S = \{\}$)
 - At each step:
 - Add the feature that gives the **best improvement**
 - Stop when:
 - Desired number of features is reached, or
 - No significant improvement occurs
-

4. Algorithm Steps

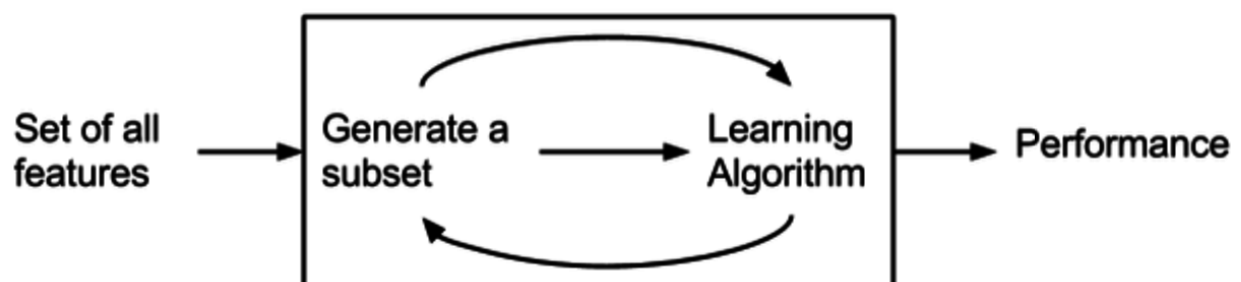
1. Start with **empty feature set** ($S = \{\}$)
 2. For each feature not in (S):
 - Temporarily add it to (S)
 - Evaluate model performance
 3. Select the feature that gives **best performance**
 4. Add it permanently to (S)
 5. Repeat steps 2–4 until stopping condition is met
-

5. Diagram: Step-by-Step Feature Addition

1st iteration 2nd iteration 3rd iteration Result



Selecting the best subset



6. Example (Conceptual)

Features = {F1, F2, F3, F4}

- Step 1: Try each → choose best → {F2}
- Step 2: Try adding remaining:
 - {F2, F1}, {F2, F3}, {F2, F4}
 - choose best → {F2, F3}
- Step 3: Continue until stopping

7. Characteristics

- **Greedy approach** (locally optimal choice)
 - Does not reconsider removed features
 - Simple and easy to implement
-

8. Advantages

- Computationally efficient
 - Easy to understand
 - Works well for small to medium datasets
-

9. Disadvantages

- May not find **global optimal solution**
 - Cannot remove features once added
 - Sensitive to feature interaction
-

10. Applications

- Machine learning model improvement
- Pattern recognition
- Data preprocessing
- Dimensionality reduction

Sequential Backward Selection (SBS)

1. Introduction

Sequential Backward Selection is a **greedy feature selection technique** that works in the **reverse direction of forward selection**.

- Starts with **all features**
- Removes features **one by one**

- At each step, removes the feature whose removal **least affects performance**
-

2. Problem Context

When a dataset has many features:

- Some features may be **irrelevant or redundant**
 - Goal → find a **smaller subset** without losing important information
 - Avoid checking all (2^n) possible subsets
-

3. Basic Idea

- Begin with full feature set ($S = \{F_1, F_2, \dots, F_n\}$)
 - At each step:
 - Remove one feature
 - Choose the removal that **minimally reduces performance**
 - Continue until stopping condition is met
-

4. Algorithm Steps

1. Start with **all features**
 2. For each feature in the set:
 - Remove it temporarily
 - Evaluate model performance
 3. Identify the feature whose removal gives **best (or least degraded) performance**
 4. Remove that feature permanently
 5. Repeat steps 2–4 until:
 - Desired number of features is reached, or
 - Performance drops significantly
-

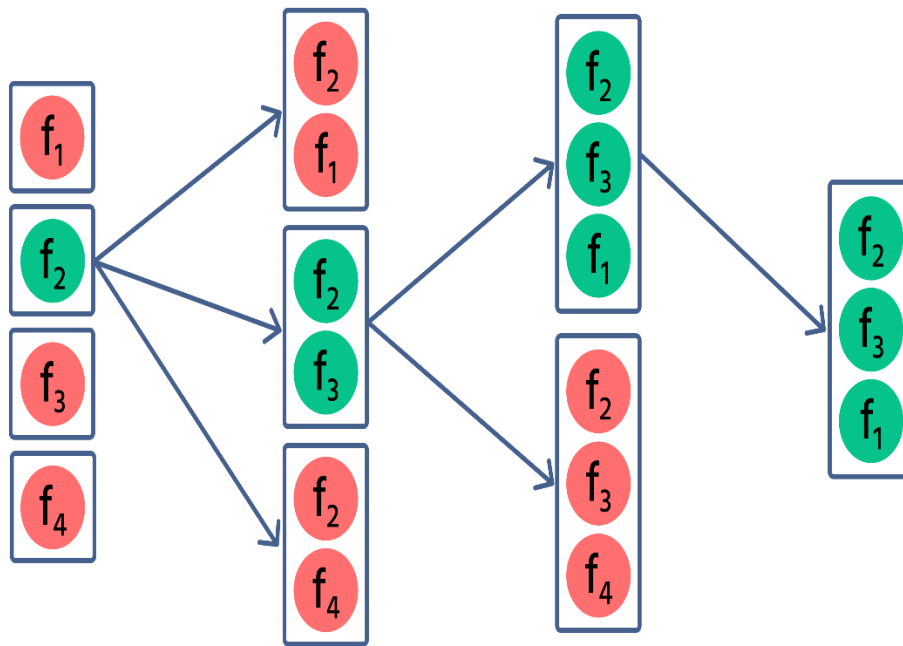
5. Diagram: Step-by-Step Feature Removal

1st iteration

2nd iteration

3rd iteration

Result



Backward stepwise selection example with 5 variables:

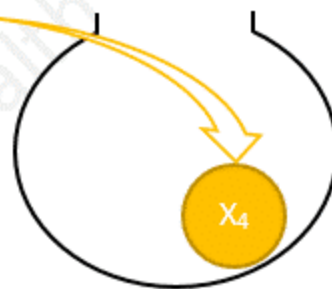
Start with a model that contains all the variables

Full Model



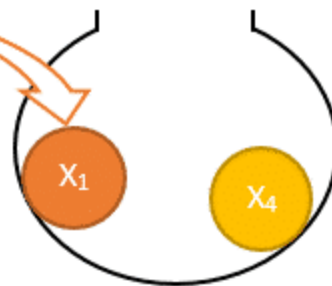
Remove the least significant variable

Model with 4 variables



Keep removing the least significant variable until reaching the stopping rule or running out of variables

Model with 3 variables



6. Example (Conceptual)

Features = {F1, F2, F3, F4}

- Step 1: Try removing each:

- Remove F1 → evaluate
 - Remove F2 → evaluate
 - Remove F3 → evaluate
 - Remove F4 → evaluate
 - Remove worst feature → {F1, F2, F3}
 - Step 2: Repeat:
 - Remove next least useful → {F1, F3}
-

7. Characteristics

- **Greedy approach**
 - Works top-down (full → reduced set)
 - Considers interaction among features initially
-

8. Advantages

- Considers **all features initially**
 - Often better when features are interdependent
 - Simple to implement
-

9. Disadvantages

- Computationally expensive for large feature sets
 - Cannot re-add removed features
 - May not give global optimal solution
-

10. Applications

- Machine learning preprocessing
- Pattern recognition
- Data mining
- Feature reduction in high-dimensional data

Cauchy–Schwarz Inequality

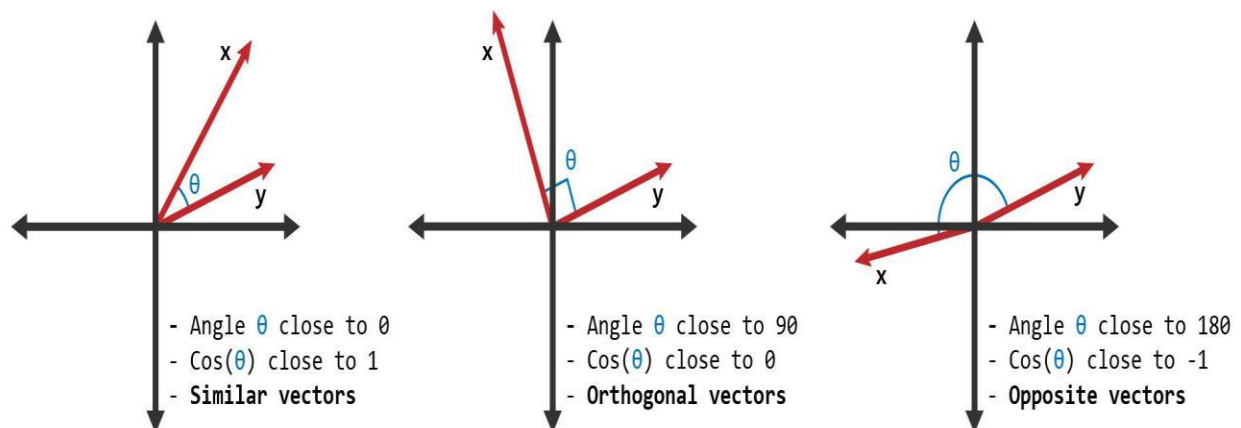
The **Cauchy–Schwarz inequality** is an important mathematical concept used in **pattern recognition, machine learning, and linear algebra**. It provides a relationship between the **dot product of two vectors** and the **product of their magnitudes**. For any two feature vectors (x) and (y), the inequality is expressed as:

Expressed as:

$$|x \cdot y| \leq \|x\| \cdot \|y\|$$

This means that the absolute value of the dot product of two vectors is always **less than or equal to** the product of their lengths (norms). This property ensures that the similarity between two vectors has an upper bound, which is very useful in analyzing patterns.

Geometric Interpretation



Geometrically, the inequality is closely related to the **angle between two vectors**. The dot product can also be written as:

$$x \cdot y = \|x\| \|y\| \cos \theta$$

where (θ) is the angle between the vectors. Since the value of ($\cos \theta$) always lies between **-1 and 1**, the inequality naturally follows. When the vectors point in the same direction, the angle is (0 degree) and the dot product is maximum. When they are perpendicular ((90 degree)), the dot product becomes zero, indicating no similarity. If they point in opposite directions ((180 degree)), the similarity becomes negative.

Role in Pattern Recognition

In pattern recognition, data is represented as **feature vectors** in a multidimensional space. The Cauchy–Schwarz inequality plays a key role in measuring **similarity between patterns**. One of its most important applications is in **cosine similarity**, which is defined as:

$$\text{Cosine Similarity} = \frac{x \cdot y}{\|x\| \|y\|}$$

Due to the Cauchy–Schwarz inequality, this value is always bounded between **-1 and 1**, ensuring stable and meaningful comparisons between data points. This is widely used in applications such as **text classification, image recognition, clustering, and recommendation systems**.

Importance in Feature Analysis

The inequality helps in:

- Ensuring that similarity measures remain within a valid range
- Comparing feature vectors irrespective of their magnitude
- Supporting algorithms like **k-NN, SVM, and clustering techniques**
- Providing mathematical foundation for optimization and error minimization

It also prevents misleading results where large magnitude vectors could otherwise dominate similarity calculations.

In simple terms, the Cauchy–Schwarz inequality tells us that:
“The similarity between two patterns cannot exceed a certain limit.”

If two feature vectors are very similar, their cosine similarity will be close to **1**. If they are completely different, it will be close to **0**, and if they are opposite, it will be **negative**.

Conclusion

The Cauchy–Schwarz inequality is a fundamental tool in pattern recognition that ensures **bounded similarity, stable computations, and meaningful comparison of feature vectors**. It forms the theoretical basis for many similarity measures and plays a crucial role in designing efficient and reliable pattern recognition systems.

Numerical Question

Given two feature vectors:

$$x = (1, 2, 3), \quad y = (4, 5, 6)$$

1. Verify the **Cauchy–Schwarz inequality**
 2. Find the **cosine similarity**
-

Step 1: Find Dot Product

$$\begin{aligned} x \cdot y &= (1 \times 4) + (2 \times 5) + (3 \times 6) \\ &= 4 + 10 + 18 = 32 \end{aligned}$$

Step 2: Find Magnitudes

$$\|x\| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14}$$

$$\|y\| = \sqrt{4^2 + 5^2 + 6^2} = \sqrt{77}$$

Step 3: Verify Cauchy–Schwarz Inequality

$$|x \cdot y| \leq \|x\| \cdot \|y\|$$

$$32 \leq \sqrt{14} \times \sqrt{77}$$

$$= \sqrt{1078} \approx 32.83$$

✓ $32 \leq 32.83 \rightarrow$ Condition satisfied

Step 4: Cosine Similarity

$$\text{Cosine Similarity} = \frac{32}{\sqrt{14} \times \sqrt{77}}$$

$$= \frac{32}{32.83} \approx 0.975$$

Final Answer

- Cauchy–Schwarz inequality is **verified** ✓
 - Cosine similarity $\approx$ **0.975** (very high similarity)
-

Interpretation

🔗 Since value is close to **1**, vectors are:

- Highly similar
- Nearly in same direction

Practice Questions:-

$$X = (2, 1) \quad y = (1, -2)$$

Find:

1. Dot product
2. Magnitudes
3. Verify inequality
4. Cosine similarity

 **Given**

$$x = (2, 1), \quad y = (1, -2)$$

 **Step 1: Dot Product**

$$\begin{aligned} x \cdot y &= (2 \times 1) + (1 \times -2) \\ &= 2 - 2 = 0 \end{aligned}$$

 **Step 2: Magnitudes**

$$\begin{aligned} \|x\| &= \sqrt{2^2 + 1^2} = \sqrt{5} \\ \|y\| &= \sqrt{1^2 + (-2)^2} = \sqrt{5} \end{aligned}$$



Step 3: Verify Cauchy–Schwarz Inequality

$$|x \cdot y| \leq \|x\| \cdot \|y\|$$

$$0 \leq \sqrt{5} \times \sqrt{5} = 5$$

✓ $0 \leq 5 \rightarrow$ Condition satisfied

Step 4: Cosine Similarity

$$\text{Cosine Similarity} = \frac{0}{\sqrt{5} \times \sqrt{5}} = 0$$

Final Answer

- Dot product = 0
- Inequality satisfied ✓
- Cosine similarity = 0



Feature Selection Criteria

Feature selection is the process of choosing the most relevant features from a dataset to improve classification performance and reduce complexity. This selection is guided by a **criterion function**, which evaluates how good a feature or a set of features is. A good criterion function should maximize class discrimination, reduce redundancy, and improve the accuracy of the model.

Objectives of Feature Selection Criteria

- Improve classification accuracy
 - Reduce dimensionality
 - Remove irrelevant and redundant features
 - Decrease computational cost
-

Types of Feature Selection Criteria

1. Probabilistic Criterion

The probabilistic approach is based on probability theory and focuses on selecting features that maximize the probability of correct classification or minimize classification error. It uses class-conditional probability distributions and Bayesian decision rules.

Key Points:

- Based on ($P(\omega_i | x)$) (posterior probability)
- Goal: Minimize Bayes error
- Requires estimation of probability density functions (PDFs)
- Works well when distributions are known

Advantages:

- High classification accuracy

Limitations:

- Computationally complex
 - Difficult to apply in high-dimensional data
-

2. Separability-Based Criterion

This approach selects features based on how well they separate different classes in the feature space. The idea is to maximize the distance between classes and minimize the spread within classes.

Key Points:

- Focus on class distance and variance
 - Uses statistical measures
 - Does not require probability estimation
-

Important Measure: Fisher Criterion

Important Measure: Fisher Criterion

$$J = \frac{(m_1 - m_2)^2}{s_1^2 + s_2^2}$$

Where:

- m_1, m_2 = class means
- s_1^2, s_2^2 = variances

Higher value of J indicates better feature separability.

Comparison

Feature	Probabilistic	Separability
Based on	Probability	Distance
Focus	Accuracy	Class separation
Complexity	High	Low
Example	Bayes classifier	Fisher criterion

Conclusion

Feature selection criteria help in identifying the most informative features for pattern recognition. Probabilistic methods aim at improving classification accuracy using probability models, while separability-based methods focus on maximizing the distinction between classes. Both approaches are important and are selected based on the problem requirements.

One-Line Memory Trick

Probabilistic = Probability decides
Separability = Distance decides

Inter-Class Distance Based Feature Selection

Inter-class distance based feature selection is a type of **separability-based criterion** used in pattern recognition. In this approach, features are selected based on how far apart different classes are in the feature space. The main idea is that a good feature should make the **distance between classes as large as possible**, so that the classifier can easily distinguish between them.

Basic Concept

In a dataset with multiple classes, each class has a **mean (center point)** in the feature space. The inter-class distance measures the distance between these class means. If the distance between class means is large, then the classes are well separated, and the feature is considered good.

Mathematical Representation

For two classes:

$$D = \|m_1 - m_2\|$$

Where:

- m_1, m_2 = mean vectors of two classes
- $\| \cdot \|$ = distance (usually Euclidean distance)

For multiple classes, the distance can be computed between all pairs of class means.

Key Idea

- Larger inter-class distance → Better feature
 - Smaller inter-class distance → Poor feature
-

Example

Consider two features:

- Feature 1:
Class A mean = 10, Class B mean = 12 $\rightarrow$ Distance = 2 (poor separation)
- Feature 2:
Class A mean = 10, Class B mean = 30 $\rightarrow$ Distance = 20 (good separation)

Thus, Feature 2 is better because it separates classes more clearly.

Advantages

- Simple and easy to compute
 - Helps in clear class separation
 - Computationally efficient
-

Limitations

- Does not consider spread (variance) within classes
 - May give misleading results if classes overlap
 - Not sufficient alone for complex datasets
-

Conclusion

Inter-class distance based feature selection focuses on maximizing the distance between class centers to improve classification performance. Although simple and effective, it is often combined with other measures to achieve better results in real-world applications.

Principles of Feature Extraction

1. Discriminative Power

Extracted features should clearly distinguish between different classes. Good features maximize the difference between classes and minimize confusion.

2. Dimensionality Reduction

The number of features should be reduced without losing important information. This helps in faster computation and avoids the “curse of dimensionality”.

3. Invariance

Features should be invariant to changes such as:

- Rotation
- Scaling
- Translation

This ensures that the system recognizes patterns even if they appear in different forms.

4. Robustness

Features should be resistant to noise and distortions in the data. Small changes in input should not significantly affect the extracted features.

5. Minimal Redundancy

Extracted features should not carry duplicate information. Each feature should contribute uniquely to the representation.

6. Computational Efficiency

The feature extraction process should be simple and fast so that it can be applied to large datasets efficiently.

7. Preservation of Important Information

While reducing data, important structural and statistical information must be retained to ensure accurate classification.